Can Language Models Postprocess Rewards for Reinforcement Learning?

Juyong Lee [1 *] **Doohyun Lee** [1 *]

Abstract

We investigate whether a pre-trained large language model can postprocess reward signals for reinforcement learning. Rewarding with visual-language alignment score from vision-language models is an attractive alternative to reward designing by experts or reward learning from human feedback. However, the quality of the reward signals throughout a trajectory is often observed to be highly noisy, becoming a bottleneck for the stable behavior learning. In this work, we exploit the code generation ability of the large language models for postprocessing the reward signals and compare the results of training with heuristic methods. We further study a simple method of Chain-of-Function which iteratively develops the generated functions given the training history. We report that LLM can suggest a new combination of existing heuristic functions, yet we do not observe improved learning results.

1. Introduction

Reinforcement learning (RL) has been a strong framework for learning complex behaviors in diverse domains, including video games (Mnih et al., 2015; Lee et al., 2022), robotic agent locomotion (Haarnoja et al., 2018; Smith et al., 2023), and image generation aligned with human preference (Fan et al., 2023; Black et al., 2023). However, despite the promise of RL,,designing a reward function remains a challenge. Currently, several attempts have shown that the reward design can be partially or even fully automated with the help of foundation models. For example, Ma et al. (2023) shows that the large language models (LLM) can generate the codes for reward function, and Fan et al. (2022) demonstrates that the alignment score between the agents' behavior and current environmental context, computed by vision-language models (VLM), can be used as rewards.

In this work, we focus on using the visual-language alignment (VLA) scores as rewards for RL. A pre-trained VLM enables VLA scores to guide an RL agent for the task described in natural language text. This can be highly beneficial when the reward is difficult to define in formal language. For example, defining a reward function for walking in a mathematical terms is difficult by specifying sequential positions of legs, while it is much easier to direct an agent to walk with visual understanding of the behavior of walking. However, one bottleneck of VLA scores comes from the high noise throughout a trajectory, as shown in Figure 2. Thus, we investigate on the method of mitigating aforementioned problem of using VLA scores as rewards.

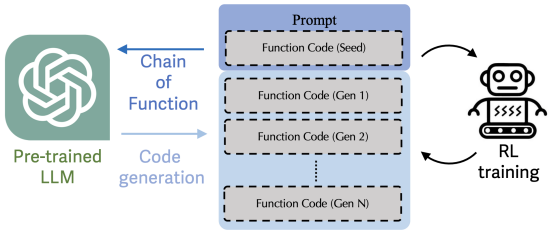

Figure 1. Chain-of-Function iteratively updates the prompt for shaping function code generation from LLM. The codes are used to shape the reward signals for training RL agents.

Here, using existing expert-designed filters is a simple yet promising solution for postprocessing. For example, moving average filter can smooth the fluctuating signals, and clipping the reward value with a threshold can help the agent to focus on high-scored parts of the trajectory. In the experiment, we observe significant improvement in success rates with heurstically chosen hyperparameters. However, exploring the space of functions with multiple hyperparameters is extremely complex, making the search procedure laborious.

Thus, we question whether LLM can help generate postprocessing functions, rather than designing the reward function from scratch, to replace the laborious search procedure by humans. A notable challenge lying here is high-level understanding on the postprocessing functions, including the expected results of it and relations between functions. To this end, we propose a method of Chain-of-Function, which enables evolutionary search of LLM by iteratively prompting the training results of the proposed function. An overview of this process is described in Figure 1.

[*]Equal contribution [1]KAIST AI. Correspondence to: Juyong Lee <agi.is@kaist.ac.kr>.

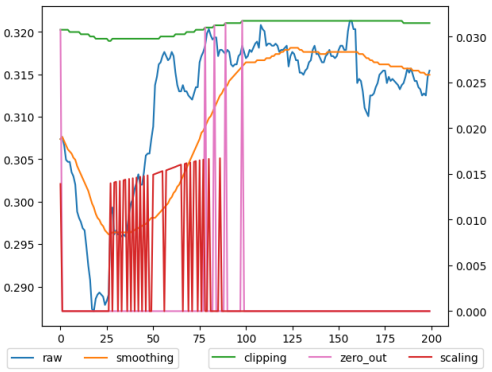

Figure 2. VLA scores from a successful trajectory in our experiment set-up. We note that the true success moment in this trajectory is at the timestep of 100. While the raw signals are highly noisy, postprocessing functions smooth the signals. The raw and smoothing signals follow left y-axis, whereas the clipping, zero-out, and scaling signals follow right y-axis. The postprocessing functions here are more described in subsection 5.3.

In summary, we study on postprocessing the direct VLA scores for RL with the follwoing points:

- we investigate the usefulness of LLM for generating the reward-postprocessing function,
- we suggest Chain-of-Function method for efficient search with LLM,
- and we examine the results of postprocessing functions generated by LLM with comparison to heuristic methods.

2. Background

2.1. Reinforcement learning with VLA Scores

Our goal is to train RL agent without reward designed by human experts but as VLA scores. For this purpose, we formulate a reward-free Markovian decision process (MDP) $(\mathcal{S}, \mathcal{A}, \mathcal{P}, \gamma)$, where $\mathcal{S}$ is the state space, $\mathcal{A}$ is the action space, $\mathcal{P}$ is the transition probability, and γ is a discount factor. Next, we define the reward signals as VLA scores from a VLM V_ϕ.

Specifically, given observations $\{o_t\}_{t=0}^T$ with a maximum trajectory leangh of T and a task description δ, the VLA scores can be computed as:

$$r_t := r(o_{\max(0,t-H):t}, \delta) = \frac{\phi_V(o_{t-H:t})^\top \phi_T(\delta)}{|\phi_V(o_{t-H:t})| \cdot |\phi_T(\delta)|},$$

where H is the window size, and ϕ_V and ϕ_T specifies the visual encoder and text encoder of V_ϕ, respectively. Using this reward, we train a policy π to maximize the expectation of total discounted rewards $\mathbb{E}_{o_t \sim \pi, \mathcal{P}} \left[\sum_t \gamma^t r(o_{t-H:t}, \delta)\right]$.

2.2. Postprocessing VLA Scores

One problem with direct VLA scores is that they are highly noisy, making RL agent be confused while training. We observe that using direct VLA scores as rewards does not result in successful training. Thus, we postprocess the reward signals to reduce the noises and emphasize the task-relevant signals with heurstically designed functions by the human experts.

We demonstrate exemplary heuristic postprocessing functions below. We display the raw signals and postprocessed signals in Figure 2. It is important to note that successful training can be achieved with the combination of introduced functions.

- Moving average: calculates the average of a specified number of neighboring scores, creating a smoothed curve by mitigating short-term fluctuations.
- Low pass filter: is another smoothing technique which only pass signals with a frequency lower than a predefined cutoff frequency and attenuate the signals with a frequency higher than the threshold.
- Clipping: bounds the signals to a predefined lower bound. This can prevent extreme values from adversely impacting the RL training.
- Zero out: refers to the process of selectively nullifying the values decreased from the previous signals, so only increasing signals survive. This approach aims to allow the RL agent to focus on pertinent part.
- Scaling: adjusts the magnitude of reward signals to a desired range or scale with a pre-defined scaling factor. This can ensure the RL agent not to be influenced by scores with significantly different magnitudes.

With a given postprocessing function f, the postprocessed VLA scores $f(r_t)$ is finally used for training our RL policy, i.e., to maximize $\mathbb{E}_{o_t \sim \pi, \mathcal{P}} \left[\sum_t \gamma^t f\big(r(o_{t-H:t}, \delta)\big)\right]$.

3. Related Work

Vision-Language Models for RL Unlike previous methodologies, our approach involves harnessing the capabilities of large-scale vision-language models, exemplified by CLIP (Radford et al., 2021), which are pretrained on extensive pairs of images and language descriptions from the internet. VLMs further extend this paradigm by integrating langauge descriptions during training. This characteristic empowers them to function as reward functions for agent learning, as showcased by recent works including MineDojo (Fan et al., 2022), DECKARD (Nottingham et al., 2023), and LAMP (Adeniji et al., 2023). Here, we investigate on the postprocessing functions for reward signals from the direct alignment score of pretrained VLMs.

LLM for code generation A paradigm shift has been witnessed, notably exemplified by pioneering contributions (Yu et al., 2023; Ma et al., 2023). These seminal works introduce a groundbreaking application of LLMs to craft and regularize reward functions that encapsulate human preferences, particularly emphasizing text-based games. In a departure from these approaches, our research takes a distinct approach by leveraging video-and-language foundation models to assess the alignment of video observations with language task specifications.

4. Method

In this section, we introduce a method of leveraging LLM for reward postprocessing. First, we explain the details of prompting LLM to generate the code for postprocessing functions. Then, we propose a method called Chain-of-Function for an efficient iterative search for LLM.

4.1. LLM for reward postprocessing function geneartion

We explore the code generation ability of LLM for postprocessing rewards. Given an LLM L_θ, our goal is to sample a reward postprocessing function $f(\cdot) : \mathbb{R} \to \mathbb{R} \sim L_\theta(f\|c)$, where c is the prompt we condition.

The prompt c consists of two parts: an instruction i and examples $\mathrm{e} = \{e_j\}$. The instruction explains brief context and goal of the task, i.e., code generation for reward postprocessing. Each example e_j is an exemplary postprocessing function f_j and corresponding results r_j of the methods.

For each result information r_j, we capture (1) success rates throughout the training of 100 epochs, where the evaluation is performed every 10 epochs, and (2) reward signals computed at the last evaluation. This evaluation is performed at the very end of training, and the reward signals are postprocessed with the postprocessing function f_j.

4.2. Chain-of-Function

To enable LLM to efficiently search over the evolution of reward postprocessing functions, we propose Chain-of-Function. At the very beginning of prompting, we build prompt c_0 with examples e_0, using heursitc functions such as moving average filtering or scaling. Then, we iteratively search the postprocessing functions with T rounds and, we call this process Chain-of-Function.

In Chain-of-Function, the prompt c_τ at round $\tau > 0$ is augmented with examples of previously sampled functions $\{f_{0,j_0}\} \cup \{f_{:\tau,j_\tau} \sim L_\theta\}$ and their corresponding training results $\{r_{0,j_0}\} \cup \{r_{:\tau,j_\tau}\}$. For better search, we suffle the order of examples and add some examples e_j^* with only functions f_j^* but without corresponding result, further discussed in subsection 5.2

It is noteworthy that this suggestion is not limited to generating postprocessing functions but can be applied to generating and type of function with LLM.

5. Experiments

In this section, we demonstrate the results from LLM-generated postprocessing functions. We show the exemplary generated function codes from LLM and compare the training results with heuristic methods.

5.1. Experimental Set-up

We train RL agent in MineDojo environment. The task description δ is to 'combat a zombie', where a zombie is randomly spawned in front of the agent. In the training phase, RL agent interacts with five randomly sampled environments. In the testing phase, RL agent gets placed in a test environment, which is different from training environments.

For the RL policy, we use PPO (Schulman et al., 2017) initialized with VPT (Baker et al., 2022), a behavior cloning policy pretrained with expert demonstrations. We note that the RL agent without VPT initialization does not learn properly. We train for 100 epochs and measure the success rate at the end of training.

For LLM L_θ, we use chatGPT(GPT3.5) (OpenAI, 2022). For the Chain-of-Function, we heuristically select two functions generated by LLM and iterate the prompt over two rounds. We also prompt for only hyperparameter search.

5.2. LLM-generated postprocessing functions

We present examples of LLM-generated postprocessing functions in Figure 3. Table 1 shows the type of postprocessing functions we use for prompting.

Since the examples in the zeroth round are in the relation of inclusion, i.e., one function overrides another example. As a result, LLM tries depth-first search only, e.g., adding normalization step at last of the latest example function.

To mitigate this sub-optimality, we shuffle the order of examples. Also, we add additional examples e_j^* with only function code f_j^*, e.g., low pass filter, but without corresponding results. With this, we observe a more diverse set of functions generated from LLM.

LLM can tune hyperparameters from the examples showcased in the prompt. For example, different values for window size parameter in smoothing or threshold value in minimum-value clipping are suggested with new postprocessing functions generated from LLM.

LLM can perform randomization and suggest a new combination of filters composed of what is provided as examples as well as not presented in the prompt. For example, LLM

```python
import numpy as np
from scipy.signal import savgol_filter

def postprocess(raw_signals,
                window_size=11, order=2, clip_percentile=90, scale=0.005):
    """Apply Savitzky-Golay filter, clip the minimum value, and scale."""
    smoothed_signal = savgol_filter(raw_signals, window_size, order)
    # Clip the minimum value based on a percentile of the original signal
    min_value = np.percentile(raw_signals, clip_percentile)
    clipped_signal = np.maximum(smoothed_signal, min_value)
    # Scale the clipped signal
    scaled_signal = scale * clipped_signal
    return scaled_signal
```

```python
import numpy as np
from scipy.signal import find_peaks

def quantumHarmonizer(raw_signals, energy_threshold=0.5):
    """
    Harness the power of quantum harmonics to enhance signal quality.

    Parameters:
    - raw_signals (numpy.ndarray): Raw input signals.
    - energy_threshold (float): Threshold for detecting peaks in the signal.

    Returns:
    - numpy.ndarray: Quantum-harmonized signals.
    """
    # Apply a quantum energy filter to identify significant peaks
    peaks, _ = find_peaks(raw_signals, height=energy_threshold)

    # Apply quantum entanglement to the signal peaks
    entangled_signals = np.sin(raw_signals) + np.cos(raw_signals)

    # Superposition of raw and entangled signals
    harmonized_signals = raw_signals + entangled_signals

    # Quantum interference to cancel noise between harmonized signals
    harmonized_signals = np.where(harmonized_signals > 0, harmonized_signals, 0)

    return harmonized_signals
```

Figure 3. Examples of postprocessing functions from LLM.

round	postprocessing function	success rate
	(raw)	0.0
0	smoothing	0.0
	clipping	0.1
	zero out	0.4
	scaling	0.8
1	normalization	0.0
	savgol filter	0.0
2	butterworth filter	0.0
	savgol exponential	0.0
3	butterworth zero out	0.0
	quantum harmonizer	0.0
+	hyperparameter	0.1

Table 1. The training results with reward postprocessing functions. We report the highest success rates over 10 rollouts averaged from 3 runs. In round 0, the postprocessing functions are designed by human experts. From round 1, the postprocessing functions are generated from LLM. (raw) means no postprocessing function applied. The round + indicates only hyperparameter search.

generates the postprocessing functions composed of shavgol filter and percentile clipping, while we do not showcase any relevant information on these choices.

Also, LLM can generate a random new filter which does not exist in the examples, as shown in Figure 3 as 'quantam harmonizer'. The actual functionality, however, is contextless, without considering actual usefulness.

LLM hallucinates the expected results of the training with generated postprocessing functions. LLM anticipates that the generated postprocessing function will achieve up to 0.9 success rates, although it is completely false (not displayed).

5.3. Comparison with heuristic methods

We compare the results of training, as demonstrated in Table 1. The detailed explanation on the postprocessing functions designed by huamn expert are as follows:

- smoothing: applying moving average smoothing on raw signals
- clipping: apply clipping with minimum value with threshold after 'smoothing'
- zero out: apply zeroing out the decreased signals after 'clipping'
- scaling: scale the reward signals (by multiplying a scale value) after 'zero out'

Below are detailed explanations of the postprocessing functions generated from LLM.

- normalization: noramlization added after 'scaling'
- savgoal filter: applying savgol filter to raw signals followed by clipping with threshold of 90 percentile and scaling
- butterworth filter: applying butterworth filter follwed by clipping with threshold of 0.9 of maximum value and scaling
- savgol exponential: applying exponential functions after savgol filter and normalization
- butterworth zero out: applying butterworth filter follwed by clipping with threshold of 0.9 of maximum value, scaling, and zero out with random criteria.
- quantum harnomizer: applying cosine and sine function to the raw signal and adding them, followed by min clipping with threshold value of 0.
- hyperparameter: this postprocessing function is generated from prompt, only asking for hyperparameter search on top of 'scaling' baseline.

Unfortunately, we do not observe any improvement of training with LLM-generated codes. Figure 4 shows the signals applied with postprocessing functions generated from LLM.

The agents trained with postprocessing function hyperparameters tuned by LLM, but without the introduction of new types of filters or algorithms, sometimes succeed on test tasks. This suggests the possibility of building efficient search system accelerated by LLM, instead of brute methods.

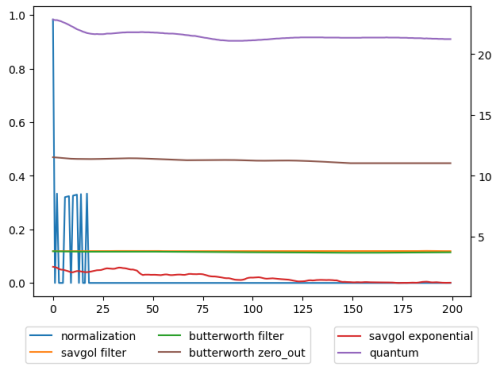

Figure 4. VLA scores postprocessed from shaping functions generated from LLM. The signals in the left caption box follows left y-axis, and the signals in the right caption box follows right y-axis.

6. Conclusion & Future Work

In this work, we investigate the code generating capability of LLM for postprocessing reward signals. We observe that LLM can generate diverse new functions. Unfortunately, however, we do not observe an increase in task success.

We note that the number of search here is limited, while scaling the number of search is not difficult. In this regard, we believe analyzing how one can guarantee the convergence of algorithm generation from LLM can be an interesting topic.

Another reason is on the complexity of the task. We witness that formulating the search space and providing good exemplary seeds in the prompt are the key to success. However, we argue that the task we target here is too complex, due to open-endedness and high dimensionality of the visual and action space. The comparison between 'scaling' and 'hyperparameter' in the experiment also implies how sensitive the RL training is to hyperparmeters.

Lastly, we anticipate that the information that we provide for prompting is limited. Although we prompt main information crucial for determining or expecting the success of RL training, more imortant detailed information is missing. We conclude that LLM still struggles to understand high-level context of training, if information is not provided enough or can't be inferred easily.

7. Contributions

This paper is a technical report for a project in AI707 (Advanced Topics in Deep Reinforcement Learning), KAIST.

- **Juyong Lee:** RL training with VLA pipeline, Training with heuristics functions, LLM prompting, and LLM-generated functions analysis.
- **Doohyun Lee:** Modifying LLM prompts, LLM prompting with Chain-of-Function, and Training with LLM-generated functions.

References

Adeniji, A., Xie, A., Sferrazza, C., Seo, Y., James, S., and Abbeel, P. Language reward modulation for pretraining reinforcement learning, 2023.

Baker, B., Akkaya, I., Zhokov, P., Huizinga, J., Tang, J., Ecoffet, A., Houghton, B., Sampedro, R., and Clune, J. Video pretraining (vpt): Learning to act by watching unlabeled online videos. *Advances in Neural Information Processing Systems*, 2022.

Black, K., Janner, M., Du, Y., Kostrikov, I., and Levine, S. Training diffusion models with reinforcement learning, 2023.

Fan, L., Wang, G., Jiang, Y., Mandlekar, A., Yang, Y., Zhu, H., Tang, A., Huang, D.-A., Zhu, Y., and Anandkumar, A. Minedojo: Building open-ended embodied agents with internet-scale knowledge, 2022.

Fan, Y., Watkins, O., Du, Y., Liu, H., Ryu, M., Boutilier, C., Abbeel, P., Ghavamzadeh, M., Lee, K., and Lee, K. Dpok: Reinforcement learning for fine-tuning text-to-image diffusion models. *arXiv preprint arXiv:2305.16381*, 2023.

Haarnoja, T., Zhou, A., Hartikainen, K., Tucker, G., Ha, S., Tan, J., Kumar, V., Zhu, H., Gupta, A., Abbeel, P., et al. Soft actor-critic algorithms and applications. *arXiv preprint arXiv:1812.05905*, 2018.

Lee, K.-H., Nachum, O., Yang, M. S., Lee, L., Freeman, D., Guadarrama, S., Fischer, I., Xu, W., Jang, E., Michalewski, H., et al. Multi-game decision transformers. *Advances in Neural Information Processing Systems*, 2022.

Ma, Y. J., Liang, W., Wang, G., Huang, D.-A., Bastani, O., Jayaraman, D., Zhu, Y., Fan, L., and Anandkumar, A. Eureka: Human-level reward design via coding large language models. *arXiv preprint arXiv: Arxiv-2310.12931*, 2023.

Mnih, V., Kavukcuoglu, K., Silver, D., Rusu, A. A., Veness, J., Bellemare, M. G., Graves, A., Riedmiller, M., Fidjeland, A. K., Ostrovski, G., et al. Human-level control through deep reinforcement learning. *nature*, 2015.

Nottingham, K., Ammanabrolu, P., Suhr, A., Choi, Y., Hajishirzi, H., Singh, S., and Fox, R. Do embodied agents dream of pixelated sheep: Embodied decision making using language guided world modelling, 2023.

OpenAI. chatgpt. https://openai.com/chatgpt, 2022.

Radford, A., Kim, J. W., Hallacy, C., Ramesh, A., Goh, G., Agarwal, S., Sastry, G., Askell, A., Mishkin, P., Clark, J., Krueger, G., and Sutskever, I. Learning transferable visual models from natural language supervision, 2021.

Schulman, J., Wolski, F., Dhariwal, P., Radford, A., and Klimov, O. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.

Smith, L., Cao, Y., and Levine, S. Grow your limits: Continuous improvement with real-world rl for robotic locomotion, 2023.

Yu, W., Gileadi, N., Fu, C., Kirmani, S., Lee, K.-H., Gonzalez Arenas, M., Lewis Chiang, H.-T., Erez, T., Hasenclever, L., Humplik, J., Ichter, B., Xiao, T., Xu, P., Zeng, A., Zhang, T., Heess, N., Sadigh, D., Tan, J., Tassa, Y., and Xia, F. Language to rewards for robotic skill synthesis. *Arxiv preprint arXiv:2306.08647*, 2023.